TALLER IoT - SimuPi y Node-RED - 2026

Guía rápida: SimuPi y Node-RED en AWS EC2

Dos instancias Debian 13 (Trixie) — A: SimuPi (bóker MQTT) · B: Node-RED con Docker Compose

Antes de empezar: grupos de seguridad (Security Groups)

Las dos instancias se acceden solo desde tu IP; el bróker MQTT de SimuPi (sin usuario ni contraseña) solo desde la Instancia B. Se configuran en EC2 → Security Groups.

Instancia	Puerto	Origen permitido	Para qué
A (SimuPi)	22	Mi IP	SSH
A (SimuPi)	5000	Mi IP	Interfaz web y API de SimuPi
A (SimuPi)	1883	Security Group de la Instancia B	Bróker MQTT
B (Node-RED)	22	Mi IP	SSH
B (Node-RED)	1880	Mi IP	Node-RED y dashboard

Importante: nunca abras los puertos 1883 ni 5000 a `0.0.0.0/0`. En los comandos, el usuario de las instancias Debian es `admin`.

PARTE 1 — Instancia A: SimuPi

Paso 1: Instalar git y clonar el repositorio

Conectate por SSH a la **Instancia A** y ejecutá:

```
sudo apt update
sudo apt install -y git
```

Clonar el repositorio de SimuPi y entrar a la carpeta:

```
git clone https://github.com/seguren/simupi.git
cd simupi
```

Paso 2: Ejecutar SimuPi desde la carpeta bin/

El ejecutable para Linux está en `bin/`. Se lo lanza con `nohup` para que **siga corriendo al cerrar la sesión SSH**. Ejecutalo siempre desde la carpeta `simupi` (ahí guarda su carpeta `data/`).

```
chmod +x bin/simupi-linux-amd64
nohup ./bin/simupi-linux-amd64 > simupi.log 2>&1 &
```

Puertos: web y API en `5000`, bróker MQTT en `1883`. SimuPi arranca con un escenario por defecto; se cambia desde la interfaz web.



Cloud Computing. Cloud Robotics. 2026



Paso 3: Verificar y anotar la IP privada

Comprobá que SimuPi responde y que los dos puertos están escuchando:

```
curl http://localhost:5000/api/health
ss -ltn | grep -E ':(1883|5000) '
```

Debe responder `{"status":"ok"}` y mostrar dos líneas LISTEN. Luego obtené la **IP privada** de la Instancia A (la primera que aparece): la vas a usar en la Instancia B como servidor MQTT.

```
hostname -I
```

Desde tu navegador abrí la interfaz web: `http://IP_PUBLICA_DE_A:5000`

Administrar SimuPi

Ver el log en vivo (salir con Ctrl+C):

```
tail -f ~/simupi/simupi.log
```

Detener SimuPi:

```
pkill -f simupi-linux-amd64
```

Volver a iniciarlo (por ejemplo, después de reiniciar la instancia):

```
cd ~/simupi
nohup ./bin/simupi-linux-amd64 > simupi.log 2>&1 &
```



Cloud Computing. Cloud Robotics. 2026



PARTE 2 — Instancia B: Node-RED con Docker Compose

Paso 1: Instalar Docker y plugin Compose oficial

Conectate por SSH a la **Instancia B**. Debian 13 requiere configurar el repositorio oficial de Docker con clave firmada GPG.

1.1 Instalar paquetes base para repositorios HTTPS:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
```

1.2 Agregar la clave GPG oficial de Docker:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

1.3 Configurar el repositorio oficial para Debian (copiar el bloque completo):

```
sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Paso 1 (continuación): instalar Docker Engine y Compose

1.4 Instalar Docker Engine y Docker Compose:

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

1.5 Habilitar la ejecución sin sudo:

```
sudo usermod -aG docker $USER
```

Ahora ejecutá este comando **solo** (abre una nueva sesión con el grupo aplicado):

```
newgrp docker
```

Validación: verificá la instalación con `docker --version` y `docker compose version` antes de avanzar.

```
docker --version
docker compose version
```



Paso 2: Crear el directorio persistente y configurar permisos

El contenedor corre con el usuario sin privilegios `node-red` (UID 1000). La carpeta del host que se monta debe tener ese propietario para poder guardar flujos, nodos y credenciales.

```
mkdir -p ~/nodered/data
cd ~/nodered
sudo chown -R 1000:1000 data
```

Paso 3: Crear el archivo docker-compose.yml

Ejecutando el bloque completo desde `~/nodered` se crea el archivo con límite de memoria y volumen persistente (sin necesidad de abrir un editor):

```
cat > docker-compose.yml <<'EOF'
services:
  nodered:
    image: nodered/node-red:latest
    container_name: nodered
    restart: unless-stopped
    ports:
      - "1880:1880"
    environment:
      - TZ=America/Argentina/Buenos_Aires
    volumes:
      - ./data:/data
    deploy:
      resources:
        limits:
          memory: 512M
EOF
```

Validá el archivo (si no muestra errores, es correcto). El acceso al puerto 1880 se controla con el Security Group de la instancia.

```
docker compose config -q
```

Paso 4: Iniciar el contenedor y verificar logs

Levantá el servicio en segundo plano e inspeccioná el arranque de Node-RED:

```
docker compose up -d
docker compose ps
docker compose logs -f
```

Salí de los logs con Ctrl+C (el contenedor sigue corriendo). Cuando aparezca `Started flows`, Node-RED está listo.



Cloud Computing. Cloud Robotics. 2026



Paso 5: Instalar el dashboard y abrir Node-RED

Instalá el nodo del dashboard dentro del contenedor y reinicialo:

```
docker exec -w /data nodered npm install @flowfuse/node-red-dashboard
docker restart nodered
```

Abrí en tu navegador el editor: `http://IP_PUBLICA_DE_B:1880` y el dashboard en `http://IP_PUBLICA_DE_B:1880/dashboard`.

Paso 6: Conectar Node-RED con SimuPi

Reemplazá `IP_PRIVADA_DE_A` por la IP privada que anotaste en la Instancia A y probá que el puerto MQTT es alcanzable desde la Instancia B:

```
timeout 3 bash -c '</dev/tcp/IP_PRIVADA_DE_A/1883' && echo CONECTA || echo SIN_CONEXION
```

Si dice `CONECTA`, en Node-RED configurá el bróker MQTT con **Server = IP privada de A** y **Port = 1883** (sin usuario ni contraseña). Si dice `SIN_CONEXION`, revisá el Security Group de A (puerto 1883 desde el SG de B) y que SimuPi esté corriendo.

Comandos útiles y respaldo

Reiniciar, ver logs recientes o detener Node-RED (desde ~/nodered):

```
cd ~/nodered
docker compose restart
docker compose logs --tail 50
docker compose down
```

Respaldar flujos y configuración (todo queda en la carpeta data/):

```
tar -czvf backup_nodered.tar.gz ~/nodered/data
```



Cloud Computing. Cloud Robotics. 2026

